Регуляризация модели

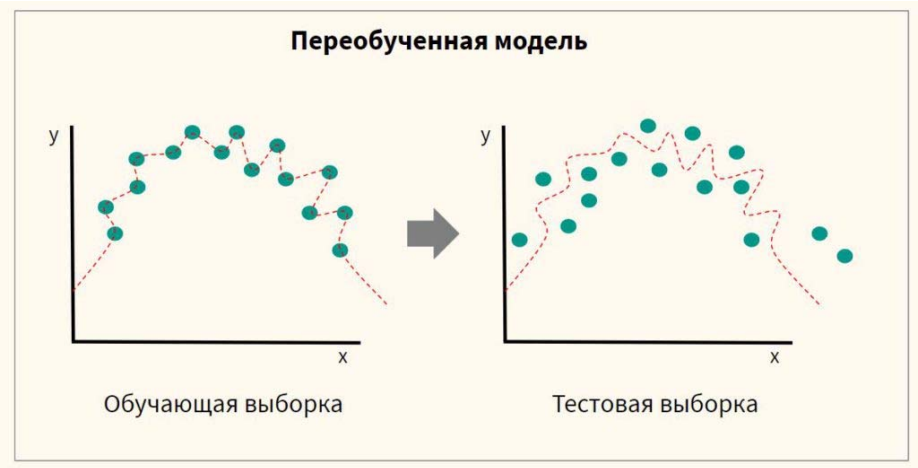
В рамках вводного курса мы обсудили вопрос разделения выборки на обучающую и тестовую части, а также затронули проблему переобучения и недообучения модели. Рассмотрим этот вопрос более подробно.

Переобучение и недообучение модели

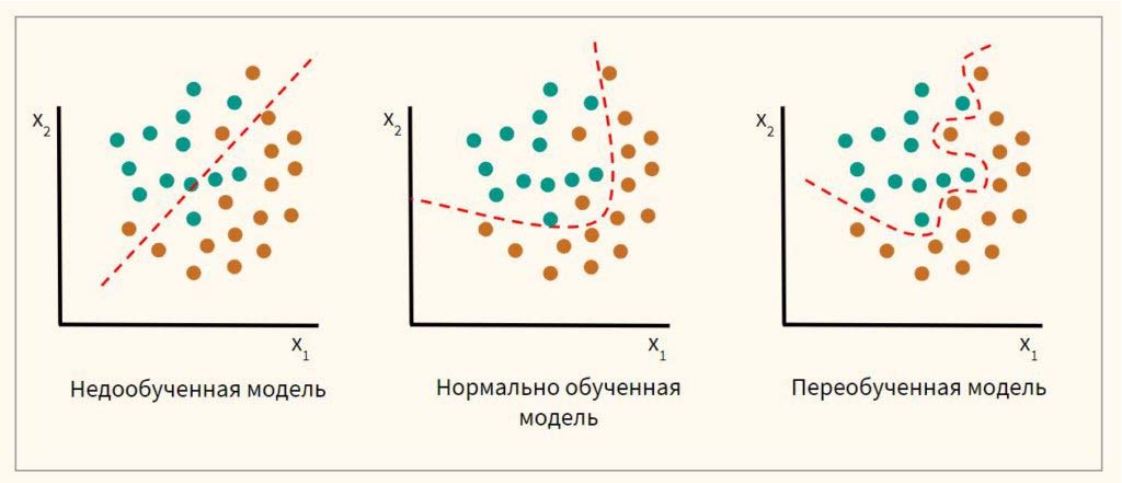
Переобучение (overfitting) случается, когда модель показывает хороший результат на обучающей выборке и сильно «проседает» на тестовых данных. При **недообучении** (underfitting) наоборот у алгоритма не получается уловить зависимость на обучающих данных.

Вспомним график из вводного курса.

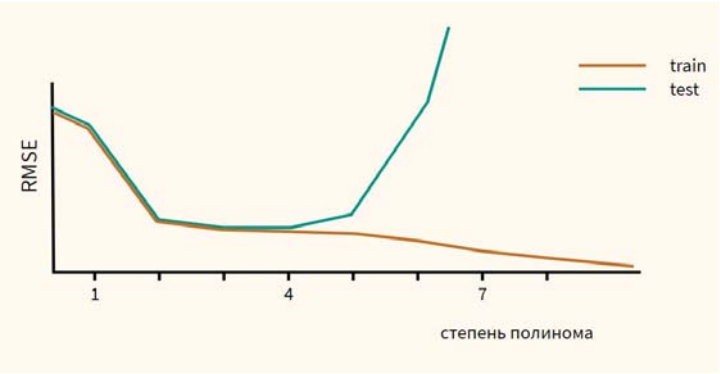
Дополним его для наглядности иллюстрацией того, как может повести себя переобученная модель на тестовых данных.



Также рассмотрим пример переобучения и недообучения в задаче классификации.



Приведем еще один график. Предположим, что мы взяли полиномиальную регрессию и смотрим на значение функции потерь на обучающих и тестовых данных для различных степеней полинома.



Как вы видите, после определенной границы *сложности модели* ошибка на тестовых данных начала расти. Рассмотрим этот вопрос под разными углами и введем несколько новых терминов.

Сложность модели

Причиной пере- и недообучения может быть степень сложности используемой модели (model complexity). Под сложностью в данном случае понимается тип или типы используемых алгоритмов, способность к выявлению линейных или нелинейных зависимостей, количество рассматриваемых признаков и другие факторы.

> В целом, можно сказать, что более простая модель склонна к недообучению, более сложная — к переобучению.

Например, полиномиальная модель (особенно с полиномом высокой степени) зачастую способна уловить более сложную зависимость, чем модель линейной регрессии. Если модель хорошо описывает исходные данные, говорят, что у нее низкое смещение.

Смещение

Смещение (bias) представляет собой разность между истинным и прогнозным значением.

$$Bias = \mathbb{E}[\hat{f}(x) - f(x)]$$

Иными словами, смещение показывает, насколько хорошо модель обучилась на имеющихся данных (настроилась на выборку).

Впрочем, как мы уже сказали в самом начале, идеально обученная (переобученная) модель может иметь минимальное смещение и при этом показать слабые результаты на тестовых данных. В этом случае говорят о большом разбросе.

Разброс

Разброс (variance) показывает насколько результат модели на обучающей выборке отличается от результата модели на тестовой выборке.

Другими словами, предсказания на тесте (например, на нескольких выборках) имеют разброс, и нам важно, чтобы эти результаты не слишком отличались (тогда можно сказать, что у модели хорошая обобщающая способность).

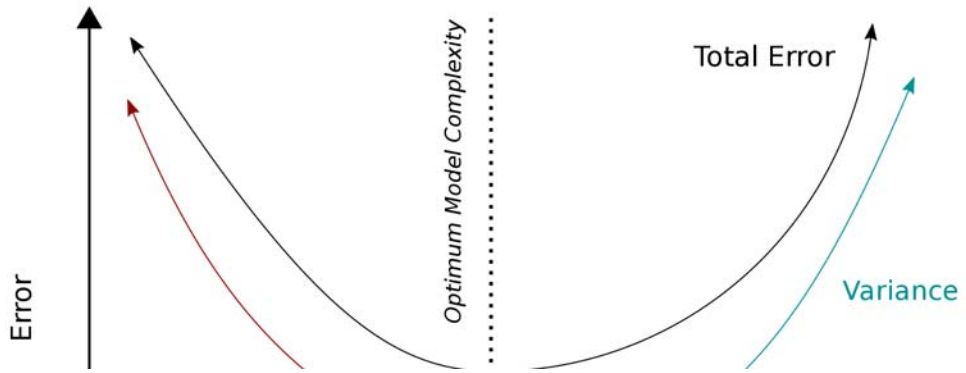
Математически, речь идет о дисперсии прогнозных значений.

$$Variance = Var(\hat{y}) = \mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right]$$

Компромисс смещения и разброса

На практике стремление к снижению смещения может привести к увеличению разброса, т.е. переобучению, и, наоборот, высокое смещение модели может способствовать низкому разбросу, но смысла в этом не будет, потому что модель будет недообучена.

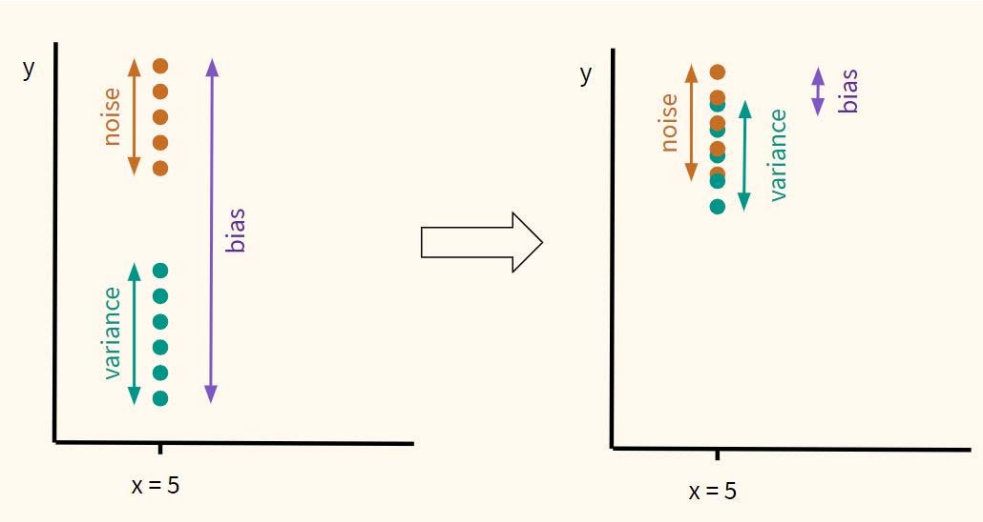
Эту проблему принято называть компромиссом или дилеммой смещения и разброса (bias-variance tradeoff). Посмотрим на рисунок ниже.



Источник: Wikipedia

Рассмотрим еще одну полезную для понимания смещения и разброса иллюстрацию. На **левом графике** изобразим модели (каждая точка на графике — это прогноз одной модели), которые не очень хорошо описывают данные. Например, представим, что мы пытаемся описать полиномом первой степени зависимость, которая гораздо лучше описывается с помощью квадратичной функции.

Оценим прогнозы этих моделей в условной точке $x = 5$.



Коричневым цветом обозначены те «правильные» ответы, которые мы хотели бы получить. Свести их в одну точку мы не можем, потому что даже в идеальном случае мы можем получить прогноз с точностью до **шума** (случайных колебаний).

Bias (фиолетовая стрелка) отражает ошибку, связанную с тем, что мы ошиблись с выбором модели (например, степенью полинома). **Разброс** (зеленый цвет) — прогнозы моделей на нескольких обучающих выборках.

На правом графике, благодаря правильно подобранным (например, полином второй степени, снижает bias) и хорошо настроенным (снижает variance) моделям мы смогли существенно улучшить качество ответов.

Шум

Шум представляет собой дисперсию истинных значений.

$$\sigma_\varepsilon^2 = Var(y) = \mathbb{E}\left[(f(x) - \mathbb{E}[f(x)])^2\right]$$

Разложение ошибки на квадрат смещения, разброс и шум

Изначально мы хотим предсказать следующую зависимость.

$$\hat{f}(X) = f(X) + \varepsilon, \varepsilon \sim N(0, \sigma_\varepsilon)$$

Ошибка в точке x равна

$$Err(x) = \mathbb{E}[(f(x) - \hat{f}(x))^2]$$

Эту ошибку можно разложить на

$$Err(x) = Bias^2 + Variance + Noise$$

$$= \left(\mathbb{E}[\hat{f}(x) - f(x)]\right)^2 + \mathbb{E}\left[\left(\hat{f}(x) - \mathbb{E}[\hat{f}(x)]\right)^2\right] + \sigma_\varepsilon^2,$$

Обоснуем это разложение. Прежде всего вспомним, что дисперсия равна матожиданию квадрата случайной величины минус квадрат ее матожидания.

$$Var(X) = \mathbb{E}[(X - \mathbb{E}[X])^2] = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

Обозначим истинные значения как y, прогнозные как $\hat{y}$.

$$Err(x) = \mathbb{E}(y - \hat{y})^2 = \mathbb{E}(y^2 + \hat{y}^2 - 2y\hat{y})$$

$$= \mathbb{E}(y^2) + \mathbb{E}(\hat{y})^2 - 2(\mathbb{E}y\hat{y})$$

Введем новые слагаемые $(\mathbb{E}y)^2 - (\mathbb{E}y)^2$ и $(\mathbb{E}\hat{y})^2 - (\mathbb{E}\hat{y})^2$.

$$Err(x) = \mathbb{E}(y^2) - (\mathbb{E}y)^2 + (\mathbb{E}y)^2 +$$

$$\mathbb{E}(\hat{y}^2) - (\mathbb{E}\hat{y})^2 + (\mathbb{E}\hat{y})^2 - 2(\mathbb{E}y\hat{y})$$

Заметим, что

$$\mathbb{E}(y^2) - (\mathbb{E}y)^2 = Var(y)$$

$$\mathbb{E}(\hat{y}^2) - (\mathbb{E}\hat{y})^2 = Var(\hat{y})$$

$$(\mathbb{E}y)^2 - 2(\mathbb{E}y\hat{y}) + (\mathbb{E}\hat{y})^2 = \mathbb{E}(y - \hat{y})^2$$

Тогда,

$$Err(x) = \underbrace{\mathbb{E}(y - \hat{y})^2}_{Bias^2} + \underbrace{Var(\hat{y})}_{Variance} + \underbrace{Var(y)}_{Noise}$$

Как бороться с переобучением?

Сложная, переобученная модель может иметь очень много признаков (особенно если речь идёт о полиноме).

Очевидно, в большинстве случае визуальный подбор степени полинома (как мы это делали ранее для линейной и логистической полиномиальных регрессий) не представляется возможным. Более того, модели с таким количеством признаков в силу их высокой сложности (complexity) склонны к переобучению.

Бороться с этим можно тремя способами:

Первый вариант. Увеличить размер обучающей выборки. Маленькая выборка снижает обобщающую способность модели, а значит повышает разброс.

Второй вариант. Уменьшить количество признаков (вручную или через алгоритм, кроме того вводить наказание за новые признаки, как например, скорректированный коэффициент детерминации, adjusted R-square). Опасность здесь — выбросить нужные признаки.

Третий вариант. Использовать регуляризацию, которая позволяет снижать параметр (вес, коэффициент) признака и, таким образом, снижать его значимость. Предположим, что мы хотим уменьшить влияние второго признака в модели ниже.

$$y = 15x_1 + 10x_2 + 5 \rightarrow y = 15x_1 + 5x_2 + 5$$

Подобное изменение, как правило, приводит к тому, что модель меньше настраивается на шум (излишнюю вариативность) в обучающих данных и, как следствие, показывает лучшие результаты на тестовой выборке.

> Другими словами, по сравнению с обычной линейной регрессией, у нее немного большее смещение (bias), но зато заметно меньший разброс (variance).

Три дополнительных соображения:

- очевидно, что если мы снизим параметр признака до нуля, то этот признак будет удален из модели;
- как правило, не имеет смысла применять регуляризацию к свободному коэффициенту θ_0;
- очень маленькие значения θ также могут повышать ошибку.

Откроем блокнот к этому занятию

boston.csv **Скачать**

Регуляризация линейной регрессии

Рассмотрим несколько вариантов регуляризации линейной регрессии: Ridge, Lasso и Elastic Net.

Подготовка данных

Подгрузим данные о недвижимости в Бостоне, разделим на обучающую и тестовую части и выполним стандартизацию.

```
boston = pd.read_csv('/content/boston.csv')
```

```python

X = boston[['LSTAT', 'RM', 'PTRATIO', 'INDUS']]
y = boston.MEDV

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = 0.3,
                                                    random_state = 42)

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

Ordinary Least Squares

Построим регрессию без регуляризации методом наименьших квадратов и выведем RMSE на обучающей и тестовой выборках.

```python
from sklearn.linear_model import LinearRegression

ols = LinearRegression()
ols.fit(X_train, y_train)

y_pred_train = ols.predict(X_train)
y_pred_test = ols.predict(X_test)

from sklearn.metrics import root_mean_squared_error

print('train: ' + str(root_mean_squared_error(y_train, y_pred_train)))
print('test: ' + str(root_mean_squared_error(y_test, y_pred_test)))
```

```
train: 5.255700309296848
test: 5.127682560624116
```

Ridge Regression

Функция потерь

Ридж-регрессия (ridge regression, гребневая регрессия) вводит наказание (penalty) за слишком большие коэффициенты. Это наказание представляет собой сумму коэффициентов θ, возведенных в квадрат (чтобы положительные и отрицательные значения не взаимоудалялись).

$$J_{Ridge}(\theta) = MSE + \sum \theta_i^2$$

Опять же θ_0 здесь не учитывается. В векторизованной форме,

$$J_{Ridge}(\theta) = MSE + \theta^T \theta$$

Корень из скалярного произведения вектора на самого себя есть его длина. Так как в данном случае мы корень не извлекаем, то можем записать слагаемое регуляризации (regularization term) как

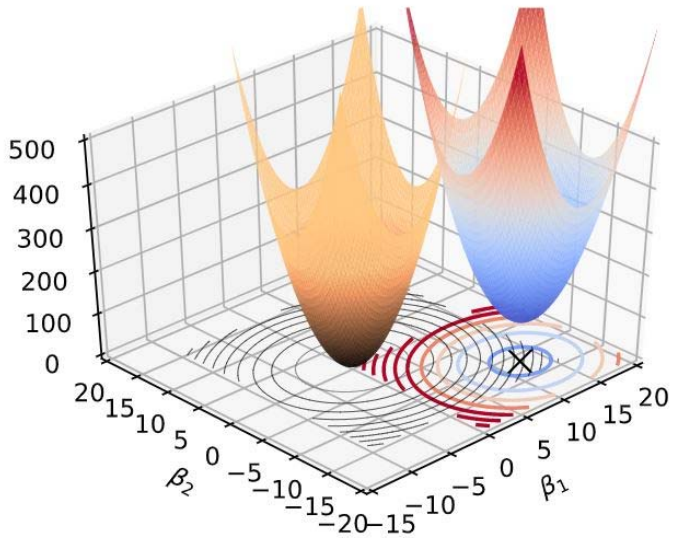
$$J_{Ridge}(\theta) = MSE + ||\theta||_2^2$$

Наконец, введем еще одно дополнение, параметр α, который позволит нам контролировать силу воздействия этого слагаемого на уровень ошибки.

$$J_{Ridge}(\theta) = MSE + \alpha \cdot ||\theta||_2^2$$

Замечу также, что Ridge Regression также называют L2-регуляризацией, потому что она использует L2-норму в качестве соответствующего слагаемого.

Посмотрим на иллюстрацию регуляризации (параметры θ заменены на β).



При регуляризации у нас появляется, по сути, две функции ошибки: основная (синяя), наказывающая за отклонение истинных значений от прогнозных, и дополнительная (оранжевая), наказывающая за отклонение коэффициентов от нуля. Их сумму и будет пытаться минимизировать алгоритм.

Источник картинки и подробности здесь.

Нормальные уравнения

Вспомним аналитическое решение задачи линейной регрессии.

$$\theta = (X^T X)^{-1} X^T y$$

В случае нормальных уравнений с L2-регуляризацией оптимальное решение можно найти через

$$\theta = (X^T X + \alpha I)^{-1} X^T y$$

I представляет собой единичную матрицу размером $(j+1) \times (j+1)$ с нулевым значением первого столбца первой строки (сдвиг), где j — количество признаков.

Что интересно, регуляризация решает проблему вырожденных или необратимых матриц. Матрица будет вырожденной в случае, если количество признаков больше количества объектов и может быть необратима, если оно равно количеству объектов. Выражение

$X^T X + \alpha I$ решает эту проблему.

Напишем собственный класс.

```
class RidgeReg():

  def __init__(self, alpha = 1.0):

    self.alpha = alpha
    self.thetas = None

  def fit(self, x, y):

    x = x.copy()
    x = self.add_ones(x)
    I = np.identity(x.shape[1])
    I[0][0] = 0

    self.thetas = np.linalg.inv(x.T.dot(x) + self.alpha * I).dot(x.T).dot(y)

  def predict(self, x):

    x = x.copy()
    x = self.add_ones(x)
    return np.dot(x, self.thetas)

  def add_ones(self, x):
    return np.c_[np.ones((len(x), 1)), x]
```

```
ridge = RidgeReg(alpha = 10)

ridge.fit(X_train, y_train)

y_pred_train = ridge.predict(X_train)
y_pred_test = ridge.predict(X_test)

print('train: ' + str(root_mean_squared_error(y_train, y_pred_train)))
print('test: ' + str(root_mean_squared_error(y_test, y_pred_test)))
```

```
train: 5.258077962476522
test: 5.104623428412015
```

Ошибка на обучающей выборке немного выросла, на тесте немного уменьшилась. Сравним с sklearn.

```
from sklearn.linear_model import Ridge

ridge = Ridge(alpha = 10)

ridge.fit(X_train, y_train)

y_pred_train = ridge.predict(X_train)
y_pred_test = ridge.predict(X_test)

print('train: ' + str(root_mean_squared_error(y_train, y_pred_train)))
print('test: ' + str(root_mean_squared_error(y_test, y_pred_test)))
```

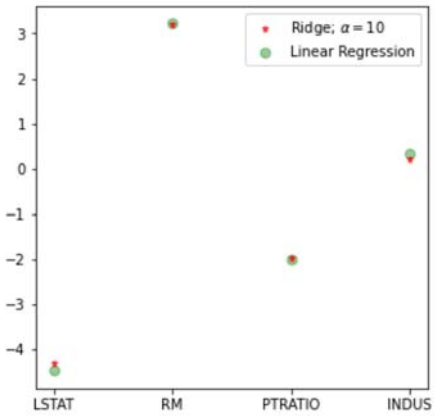
```
train: 5.258077962476521
test: 5.104623428412015
```

Выведем изменение коэффициентов на графике.

```
features = X.columns

plt.figure(figsize = (5, 5))
```

```
plt.plot(features, ridge.coef_, alpha=0.7, linestyle='none' , marker='*', markersize=5,
plt.plot(features, ols.coef_, alpha=0.4, linestyle='none', marker='o', markersize=7, cc
plt.xticks(rotation = 0)
plt.legend()
plt.show()
```



Метод градиентного спуска

Эту же задачу можно решить методом градиентного спуска. Функция потерь в этом случае будет иметь вид

$$J(\theta_j) = \frac{1}{2n}\sum(y - \theta X)^2 + \frac{1}{2n}\lambda\theta^T\theta$$

Для градиентного спуска, чтобы не запутаться, обозначим коэффициент скорости обучения через α, а параметр регуляризации через λ. При этом градиент будет равен

$$\frac{\partial}{\partial\theta_j}J(\theta) = (-x_j(y - X\theta) + \lambda\theta) \times \frac{1}{n}$$

Нормализовывать сдвиг не обязательно (в первую очередь имеют значения наклоны). Здесь мы это делаем, чтобы упростить код. Перейдем к практике. Начнем с собственного класса.

```
class RidgeGD():
  def __init__(self, alpha = 0.0001):
    self.thetas = None
    self.loss_history = []
    self.alpha = alpha

  def add_ones(self, x):
    return np.c_[np.ones((len(x), 1)), x]

  def objective(self, x, y, thetas, n):
    return (np.sum((y - self.h(x, thetas)) ** 2) + self.alpha * np.dot(thetas, thetas)

  def h(self, x, thetas):
    return np.dot(x, thetas)

  def gradient(self, x, y, thetas, n):
    return (np.dot(-x.T, (y - self.h(x, thetas))) + (self.alpha * thetas)) / n

  def fit(self, x, y, iter = 20000, learning_rate = 0.05):
    x, y = x.copy(), y.copy()
    x = self.add_ones(x)

```

```
    thetas, n = np.zeros(x.shape[1]), x.shape[0]

    loss_history = []

    for i in range(iter):
      loss_history.append(self.objective(x, y, thetas, n))
      grad = self.gradient(x, y, thetas, n)
      thetas -= learning_rate * grad

    self.thetas = thetas
    self.loss_history = loss_history

  def predict(self, x):
    x = x.copy()
    x = self.add_ones(x)
    return np.dot(x, self.thetas)
```

```
ridge_gd = RidgeGD(alpha = 0.0001)

ridge_gd.fit(X_train, y_train, iter = 50000, learning_rate = 0.01)

y_pred_train = ridge_gd.predict(X_train)
y_pred_test = ridge_gd.predict(X_test)

print('train: ' + str(root_mean_squared_error(y_train, y_pred_train)))
print('test: ' + str(root_mean_squared_error(y_test, y_pred_test)))
```

```
train: 5.255700309301131
test: 5.127682311863072
```

Сравним с sklearn. Для того чтобы оптимизировать Ridge регрессию методом градиентного спуска, используем класс SGDRegressor с параметром penalty = ‘l2’. Кроме того, укажем другие параметры для того, чтобы применяемый алгоритм был максимально близок к написанному вручную коду.

```
from sklearn.linear_model import SGDRegressor

ridge_gd = SGDRegressor(loss = 'squared_error',
                        penalty = 'l2',
                        alpha = 0.0001,
                        max_iter = 50000,
                        learning_rate = 'constant',
                        eta0 = 0.001,
                        random_state = 42)

ridge_gd.fit(X_train, y_train)

y_pred_train = ridge_gd.predict(X_train)
y_pred_test = ridge_gd.predict(X_test)

print('train: ' + str(root_mean_squared_error(y_train, y_pred_train)))
print('test: ' + str(root_mean_squared_error(y_test, y_pred_test)))
```

```
train: 5.25595467395471
test: 5.120506389637218
```

Выбор α

Подобрать оптимальный гиперпараметр регуляризации можно, в частности, с помощью класса RidgeCV, который одновременно выполнит оценку качества модели с помощью перекрестной валидации.

Приведем пример.

```
from sklearn.linear_model import RidgeCV

# укажем параметры регуляризации alpha, которые хотим протестировать
# дополнительно укажем количество частей (folds), параметр cv,
# на которое нужно разбить данные при оценке качества модели
ridge_cv = RidgeCV(alphas = [0.1, 1.0, 10], cv = 10)
ridge_cv.fit(X_train, y_train)

# выведем оптимальный параметр и достигнутое качество
ridge_cv.alpha_, ridge_cv.best_score_
```

```
(10.0, 0.6482859868459572)
```

Lasso Regression

Регрессия Lasso для регуляризации использует сумму весов по модулю, то есть L1-норму.

Сравнение L1 и L2 регуляризации

$$J_{Lasso}(\theta) = MSE + \alpha \cdot ||\theta||_1$$

Отличие заключается в том, что Ridge-регрессия при возведении θ в квадрат сильнее «наказывает» за большие коэффициенты, Lasso «наказывает» все коэффициенты одинаково. Например, если θ равен $\begin{bmatrix} 10 \\ 5 \end{bmatrix}$, то в первом случае L2-регрессии penalty составит $10^2 + 5^2 = 125$, в случае L1, только 15. При увеличении первого коэффициента на единицу L2-penalty составит $11^2 + 5^2 = 146$, L1 увеличится до 16.

Одновременно этот пример показывает особую важность масштабирования признаков при использовании алгоритма с регуляризацией.

Обнуление коэффициентов

L1 регуляризация может полностью обнулить часть коэффициентов и, таким образом, исключить соответствующие признаки из модели. Эта особенность Lasso регрессии называется parameter sparcity.

Математика

Вначале рассмотрим модель с L2-регуляризацией с одним признаком и одним коэффициентом θ.

$$L_2 = (y - x\theta)^2 + \alpha\theta^2 = y^2 - 2xy\theta + x^2\theta^2 + \alpha\theta^2$$

Найдем производную относительно θ, приравняем ее к нулю и решим для θ.

$$\frac{\partial L_2}{\partial \theta} = -2xy + 2x^2\theta + 2\alpha\theta$$

$$-2xy + 2x^2\theta + 2\alpha\theta = 0$$

$$\theta(x^2 + \alpha) = xy$$

$$\theta = \frac{xy}{(x^2 + \alpha)}$$

Коэффициент θ превратится в ноль для ненулевых значений x и y при $\alpha \to \inf$. Сравним с L1-регуляризацией. Найдем производную относительно θ, приравняем к нулю и решим для θ для случаев $\theta > 0$ (в точке ноль производная функции не определена).

$$\frac{\partial L_1}{\partial \theta} = -2xy + 2x^2\theta + \alpha$$

$$-2xy + 2x^2\theta + \alpha = 0$$

$$2x^2\theta = 2xy - \alpha$$

$$\theta = \frac{2xy - \alpha}{2x^2}$$

Сдалаем то же самое для случая, когда $\theta < 0$.

$$\frac{\partial L_1}{\partial \theta} = -2xy + 2x^2\theta - \alpha$$

$$-2xy + 2x^2\theta - \alpha = 0$$

$$2x^2\theta = (\alpha + 2xy)$$

$$\theta = \frac{\alpha + 2xy}{2x^2}$$

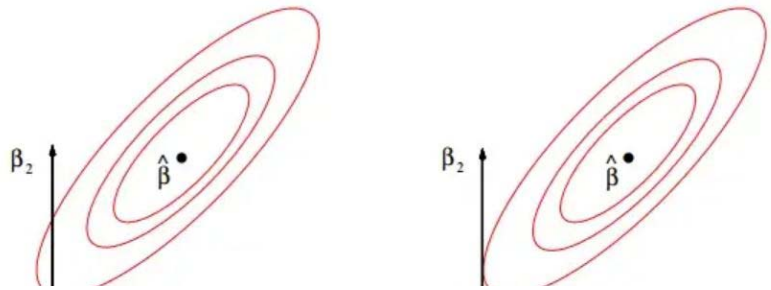
В обоих случаях при определенных x, y и α параметр θ может быть равен нулю.

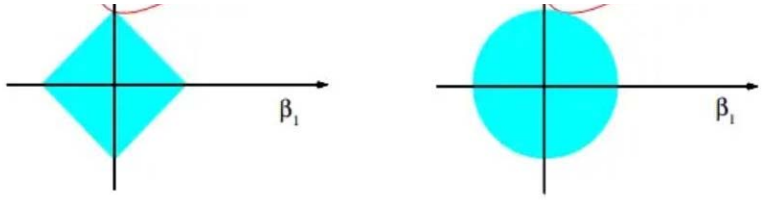
Дополнительно сравним L2 и L1 нормы двух векторов $\begin{bmatrix} 1 \\ 0 \end{bmatrix}$ и $\begin{bmatrix} \frac{1}{\sqrt{2}} \\ \frac{1}{\sqrt{2}} \end{bmatrix}$.

В случае L2-нормы, их длина будет равна единице. В случае L1-нормы длина первого вектора будет равна единице, а второго $\sqrt{2} \approx 1,41$. Таким образом, L1 регуляризация не только допускает, но и способствует обнулению коэффициентов.

Визуализация

Приведем визуализацию границ двух видов регуляризации из книги T. Hastie *The Elements of Statistical Learning*.





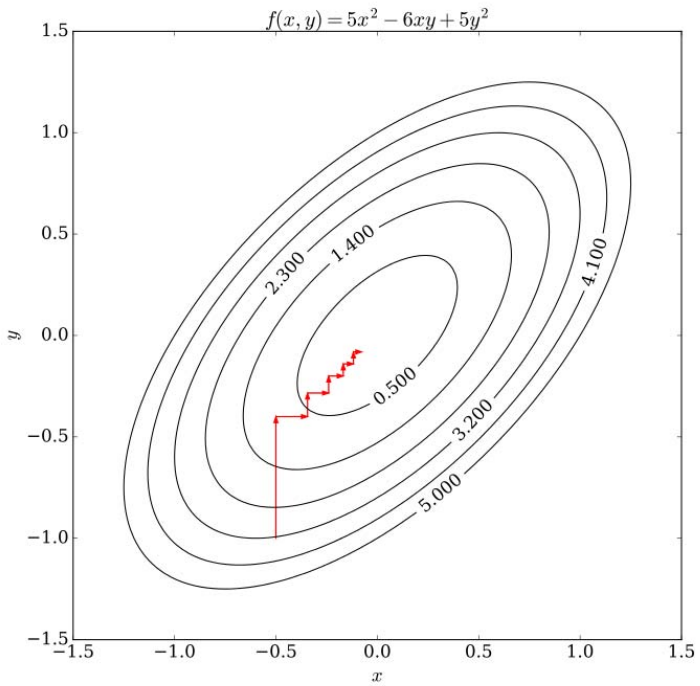
Изолинии отображают уровни ошибки при определенных параметрах θ. Синий ромб и круг задают соответственно границу L1 и L2 регуляризации.

Координатный спуск

Как уже было замечено, производная функции абсолютной величины не определена в точке $x = 0$ и, как следствие, мы не можем использовать обычный алгоритм градиентного спуска.

При этом мы можем на каждой итерации двигаться вниз вдоль одной из координат и остановиться, если этот шаг окажется меньше, чем некоторое заданное нами пороговое значение. Такой метод называется методом **координатного спуска** (coordinate descent).

Приведем иллюстрацию из Википедии.



Примечание. Обратим внимание, что в данном случае x и y обозначают веса или коэффициенты, а не данные. Такая нотация принята в литературе по оптимизации.

Выбрать, по какой координате спускаться можно разными способами, в частности, можно:

- выбирать признак (координату) случайным образом; или
- просто идти от признака 0 до j (далее признаки мы будем обозначать именно так).

Рассмотрим, как координатный спуск выглядит с точки зрения математики.

Математика

Итак, напомню, функция потерь будет иметь вид

$$J_{Lasso}(\theta) = MSE + \alpha \cdot ||\theta||_1$$

Вначале обратимся к первому компоненту, а именно MSE. Вспомним, чему равен градиент MSE.

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{n} \cdot (-x_j(y - X\theta))$$

Перепишем $y - X\theta$ как $y - (\theta_k X_k + \theta_j x_j)$.

$$\frac{1}{n} \cdot (-x_j(y - (\theta_k X_k + \theta_j x_j)))$$

Другими словами, выделим ту координату j (признак), по которой мы будем спускаться на конкретной итерации. $\theta_k X_k$ — это прогноз модели без координанты j. Раскроем, а затем реорганизуем скобки.

$$\frac{1}{n} \cdot (-x_j((y - \theta_k X_k) - \theta_j x_j))$$

Умножим $-x_j$ на каждое из слагаемых.

$$\frac{1}{n} \cdot \left(-x_j(y - \theta_k X_k) + \theta_j x_j^2\right)$$

Обозначим компоненты через новые переменные ρ («ро») и z.

$$\rho_j = x_j(y - \theta_k X_k)$$

Примечание. Минус перед x_j появится в окончательной формуле ниже.

$$z_j = x_j^2$$

> Переменная ρ показывает, насколько признак x_j коррелирует с остатками модели $y - \theta_k X_k$, построенной без j. Если ρ (т.е. корреляция) растет, значит этот признак важен для модели, и его вес стоит увеличить.

Тогда выражение будет иметь вид

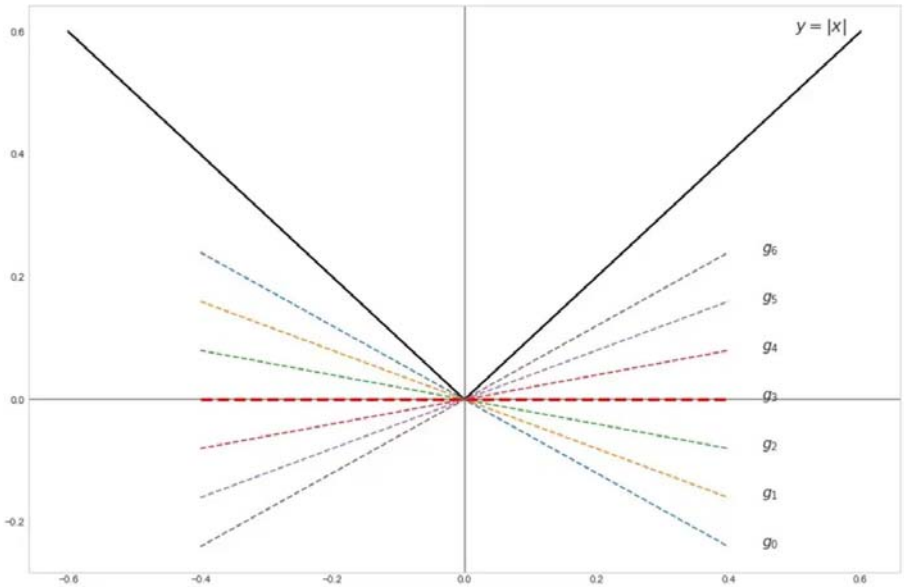
$$\frac{\partial L}{\partial \theta_j} = -\frac{1}{n}\rho_j + \frac{1}{n}\theta_j z_j$$

Приравняв производную к нулю, найдем решение относительно θ_j.

$$\theta_j = \frac{\rho_j}{z_j}$$

Замечу, что z_j, то есть скалярное произведение вектора признака x_j на самого себя, будет равно единице, если признаки будут предварительно нормализованы (приведены к единичной L2 норме).

Обратимся ко второму компоненту — слагаемому регуляризации. Так как производная в точке $|\theta_j| = 0$ не определена, воспользуемся таким понятием как **субградиент** (subgradient). По сути, субградиентом является множество возможных градиентов в определенной точке. Приведем иллюстрацию для субградиентов в точке $|\theta_j| = 0$.



Как видно, субградиенты могут принимать значения $[-1, 1]$ (градиенты функции абсолютного значения) или вернее от $[-\alpha, \alpha]$, то есть параметра, на который мы умножаем слагаемое регуляризации.

$$\alpha \frac{\partial |\theta_j|}{\partial \theta_j} = \begin{cases} -\alpha & \text{when } \theta_j < 0 \\ [-\alpha, \alpha] & \text{when } \theta_j = 0 \\ \alpha & \text{when } \theta_j < 0 \end{cases}$$

Объединим две производные.

$$\frac{\partial L(\theta)}{\partial \theta_j} = -\frac{1}{n}\rho_j + \frac{1}{n}\theta_j z_j + \begin{cases} -\alpha & \text{when } \theta_j < 0 \\ [-\alpha, \alpha] & \text{when } \theta_j = 0 \\ \alpha & \text{when } \theta_j < 0 \end{cases}$$

Рассмотрим каждый из трех случаев по отдельности.

Вариант 1. $\theta_j < 0$

$$-\frac{1}{n}\rho_j + \frac{1}{n}\theta_j z_j - \alpha = 0$$

$$\theta_j = \frac{\rho_j + n\alpha}{z_j}$$

Для того чтобы $\theta_j < 0$, необходимо чтобы $\rho_j < n\alpha$.

Вариант 2. $\theta_j = 0$

$$-\frac{1}{n}\rho_j + \frac{1}{n}\theta_j z_j + [-\alpha, \alpha] = 0$$

$$\theta_j = \left[\frac{\rho_j - n\alpha}{z_j}, \frac{\rho_j + n\alpha}{z_j}\right]$$

Для того чтобы $\theta_j = 0$, $-n\alpha \ \leq \rho_j \leq n\alpha$.

Вариант 3. $\theta_j > 0$

$$-\frac{1}{n}\rho_j + \frac{1}{n}\theta_j z_j + \alpha = 0$$

$$\theta_j = \frac{\rho_j - n\alpha}{z_j}$$

Для того чтобы $\theta_j > 0$, $\rho_j > n\alpha$.

Сдвиг θ_0 не регуляризуется и изменяется с помощью частной производной MSE (первое слагаемое).

$$\theta_0 = \frac{\rho_0}{z_0}$$

Таким образом, шаги алгоритма координатного спуска будут следующими:

Шаг 1. Добавить первый (!) столбец из единиц для коэффициента θ_0. Инициализировать веса θ;

Шаг 2. Рассчитать z_j;

$$z_j = \sum x_j^2$$

Шаг 3. Задать начальный максимальный размер шага (max_step) и пороговое значение (tolerance);

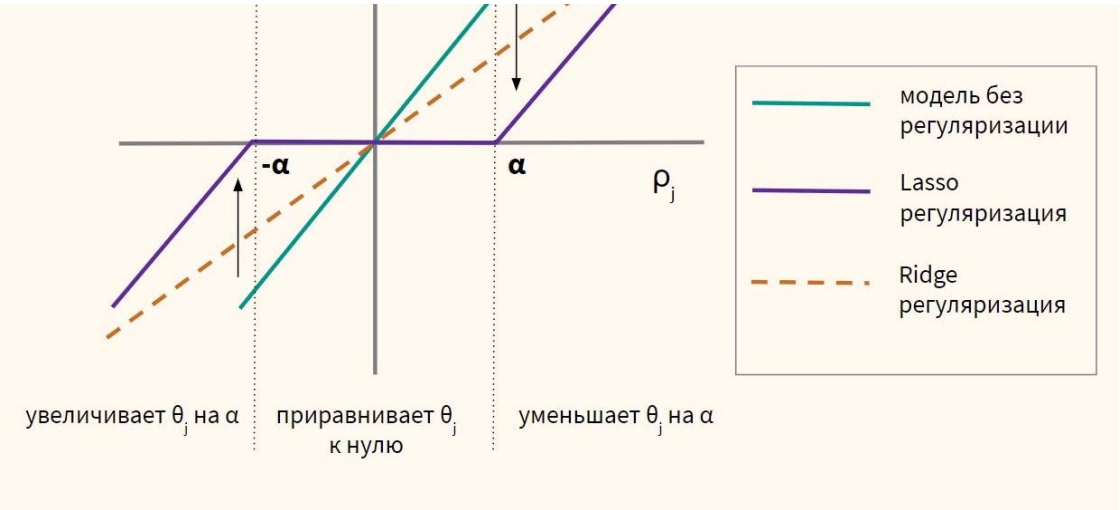
Шаг 4. Пока максимальный размер шага больше порогового значения проходиться в цикле по каждой из координат (признаку), рассчитывать $\rho_j = \sum x_j((y - \theta_k X_k)$ и обновлять вес θ этого признака в соответствии со следующими правилами.

Если $j = 0$, то есть речь идет о первом по счету коэффициенте θ_0, то $\theta_j = \frac{\rho_j}{z_j}$. В остальных случаях:

$$\theta_j = \begin{cases} \frac{\rho_j + n\alpha}{z_j} & \text{if} \quad \rho_j < -n\alpha \\ 0 & \text{if} \quad -n\alpha \leq \rho_j \leq n\alpha \\ \frac{\rho_j - n\alpha}{z_j} & \text{if} \quad \rho_j > n\alpha \end{cases}$$

Обратите внимание, если $-n\alpha \ \leq \rho_j \leq n\alpha$ (то есть корреляция признака очень мала по модулю), то коэффициент будет обнулен и исключен из модели. В двух других случаях (варианты 1 и 3), мы будем увеличивать коэффициент при большом отрицательном ρ_j и уменьшать при большом положительном (т.е. выполнять регуляризацию).

Посмотрим на график ниже.



Такое ограничение называется мягким порогом (soft thresholding). Для сравнения приведем Ridge регрессию, которая сокращает коэффициенты на всем диапазоне θ, но не приводит их к нулевым значениям.

Реализация на Питоне

Объявим вспомогательные функции.

```
# добавим столбец из единиц в матрицу признаков
def add_ones(x):
  return np.c_[np.ones((len(x), 1)), x]

# объявим функцию гипотезы
def h(x, theta):
  # 506 x (4 + 1) на (4 + 1) x 1
  return np.dot(x, theta) # 506 x 1

# рассчитаем rho_j
def rho(y, x, theta, j):
  # удалим признак (координату) j из матрицы признаков и вектора весов
  x_k = np.delete(x, j, 1)
  theta_k = np.delete(theta ,j)
  # сделаем прогноз без j
  y_pred_k = h(x_k, theta_k)
  # найдем остатки при прогнозе без j
  residuals_k = y - y_pred_k
  # вычислим rho_j
  rho_j = np.sum(x[:,j] * residuals_k)
  return rho_j

# рассчитаем z
def compute_z(x):
  # рассчитаем z (т.е. L2 норму) для каждого столбца (axis = 0)
  z_vector = np.sum(x * x, axis = 0)
  return z_vector
```

Объявим функцию координатного спуска.

```
def coordinate_descent(x, y, alpha = 0.1, tolerance = 0.0001):

  x = add_ones(x)
  theta = np.zeros(x.shape[1], dtype = float)

  z = compute_z(x)
```

```

  max_step = 100.
  iterations = 0

  # tolerance - пороговое значение, если шаг меньше, остановимся
  while(max_step > tolerance):
    iterations += 1
    old_theta = np.copy(theta)

    # поочередно выберем каждый из признаков
    for j in range(len(theta)):

      # рассчитаем rho_j
      rho_j = rho(y, x, theta, j)

      # пропишем обновление для сдвига (первый по счету коэффициент)
      if j == 0:
        theta[j] = rho_j / z[j]

      # пропишем обновления для остальных признаков
      elif rho_j < -alpha * len(y):
        theta[j] = (rho_j + (alpha * len(y))) / z[j]

      elif rho_j > -alpha * len(y) and rho_j < alpha * len(y):
        theta[j] = 0.

      elif rho_j > alpha * len(y):
        theta[j] = (rho_j - (alpha*len(y))) / z[j]

    # запишем текущий максимальный для всех координат (!!!) размер шага
    # (обратите внимание, код ниже выполняется после цикла for)
    step_sizes = abs(old_theta - theta)
    max_step = step_sizes.max()

  return theta, iterations, max_step
```

Протестируем алгоритм.

```
theta_opt, iterations, max_step = coordinate_descent(X_train, y_train,
                                                     alpha = 0.2,
                                                     tolerance = 0.0001)

np.set_printoptions(precision = 3, suppress = True)

print("Intercept is:",theta_opt[0])
print("\nCoefficients are:\n",theta_opt[1:])
print("\nNumber of iterations is:", iterations)
```

```
Intercept is: 23.01581920903955

Coefficients are:
 [-4.228  3.107 -1.811  0.   ]

Number of iterations is: 11
```

Мы видим, что модель сочла признак INDUS не имеющим значения для данной модели и обнулила соответствующий коэффициент. Сравним с классом Lasso библиотеки sklearn.

```
from sklearn.linear_model import Lasso
lasso = Lasso(alpha = 0.2, tol = 0.0001)
lasso.fit(X_train, y_train)

print("sklearn Lasso intercept :",lasso.intercept_)
print("\nsklearn Lasso coefficients :\n",lasso.coef_)
print("\nsklearn Lasso number of iterations :",lasso.n_iter_)
```

```
sklearn Lasso intercept : 23.01581920903955

sklearn Lasso coefficients :
 [-4.228  3.107 -1.811  0.   ]

sklearn Lasso number of iterations : 10
```

Elastic Net Regression

Регрессия Elastic Net использует как L1, так и L2 регуляризацию.

$$J_{ElasticNet}(\theta) = MSE + \alpha_1 \cdot ||\theta||_1 + \alpha_2 \cdot ||\theta||_2^2$$

Так как в этой модели есть компонент L1, для которого в точке $|\theta_j| = 0$ производная не определена, используем один из возможных субградиентов, а именно нулевой субградиент (красная пунктирная линия g_3 на рисунке выше).

Другими словами, субградиент для компонента будет выглядеть следующим образом

$$\frac{\partial J_{L1}}{\partial_{sub}\theta} = \begin{cases} 1 & \text{if } \theta > 0 \\ 0 & \text{if } \theta = 0 \\ -1 & \text{if } \theta < 0 \end{cases}$$

Это условие соответствует сигнум-функции или функции знака (sign function). На Питоне можно использовать **np.sign()**.

```python
class ElasticNet():
  def __init__(self, a1 = 0.1, a2 = 0.1):
    self.thetas = None
    self.loss_history = []
    self.a1 = a1
    self.a2 = a2

  def add_ones(self, x):
    return np.c_[np.ones((len(x), 1)), x]

  def objective(self, x, y, thetas, n):
    return (np.sum((y - self.h(x, thetas)) ** 2) + self.a1 * np.sum(np.abs(thetas)) +

  def h(self, x, thetas):
    return np.dot(x, thetas)

  def gradient(self, x, y, thetas, n):
    return (np.dot(-x.T, (y - self.h(x, thetas))) + (self.a1 * np.sign(thetas)) + (sel

  def fit(self, x, y, iter = 20000, learning_rate = 0.05):
    x, y = x.copy(), y.copy()
    x = self.add_ones(x)

    thetas, n = np.zeros(x.shape[1]), x.shape[0]

    loss_history = []

    for i in range(iter):
      loss_history.append(self.objective(x, y, thetas, n))
      grad = self.gradient(x, y, thetas, n)
      thetas -= learning_rate * grad

    self.thetas = thetas
    self.loss_history = loss_history
```

```python

    def predict(self, x):
        x = x.copy()
        x = self.add_ones(x)
        return np.dot(x, self.thetas)
```

```python
elastic = ElasticNet()

elastic.fit(X_train, y_train, iter = 50000, learning_rate = 0.01)

y_pred_train = elastic.predict(X_train)
y_pred_test = elastic.predict(X_test)

print('train: ' + str(root_mean_squared_error(y_train, y_pred_train)))
print('test: ' + str(root_mean_squared_error(y_test, y_pred_test)))
```

```
train: 5.255705191138267
test: 5.127348870379186
```

```python
elastic.thetas
```

```
array([23.009, -4.473,  3.219, -1.998,  0.317])
```

Сравним с классом ElasticNet библиотеки sklearn.

```python
from sklearn.linear_model import ElasticNet

elastic = ElasticNet(alpha = 0.05)
elastic.fit(X_train, y_train)

elastic.fit(X_train, y_train)

y_pred_train = elastic.predict(X_train)
y_pred_test = elastic.predict(X_test)

print('train: ' + str(root_mean_squared_error(y_train, y_pred_train)))
print('test: ' + str(root_mean_squared_error(y_test, y_pred_test)))
```

```
train: 5.259317264886122
test: 5.100827371724984
```

```python
elastic.intercept_, elastic.coef_
```

```
(23.01581920903955, array([-4.287,  3.179, -1.944,  0.146]))
```

Замечу, что класс ElasticNet библиотеки sklearn использует координатный спуск для минимизации функции потерь. Кроме того, в документации можно посмотреть, как единственный параметр α регулирует значимость L1 и L2 компонентов.

Регуляризация нейросети

В качестве иллюстрации приведем примеры L2 регуляризации нейронной сети. Реализуем эту модель на основе уже созданного алгоритма бинарной классификации со смещением с двумя признаками и одним скрытым слоем (три нейрона) и сравним результаты обучения.

Функция потерь с регуляризацией

Посмотрим на функцию потерь с учетом регуляризации (бинарная классификация)

$$J(W) = J_{\text{nn}}(W) + J_{\text{reg}}(W) =$$

$$-\frac{1}{n}\sum y \cdot log(a^{(L)}) + (1-y) \cdot log(1-a^{(L)}) +$$

$$+\frac{\lambda}{2n}\sum\sum\sum W^{(l)2}$$

Мы находим уровень ошибки на основе значений выходного слоя $a^{(L)}$. Кроме того, мы добавляем значение регуляризации, которое рассчитывается как сумма сумм весов слоев (1) и (2) и контролируется коэффициентом λ.

Приведем код.

```
# в коде обозначим коэффициент lambda как reg
def objective(y, y_pred, n, reg):

  y_one_loss = y * np.log(y_pred + 1e-9)
  y_zero_loss = (1 - y) * np.log(1 - y_pred + 1e-9)
  log_loss = - np.sum(y_zero_loss + y_one_loss) / n

  # по умолчанию np.sum() складывает по всем измерениям
  reg_loss = (np.sum(W1**2) + np.sum(W2**2)) * reg / (2 * n)

  return log_loss + reg_loss
```

Градиент компонента регуляризации для слоя l будет иметь вид

$$\frac{\partial J_{\text{reg}}}{\partial W^{(l)}}\left(\frac{\lambda}{2n}W^{(l)2}\right) = \frac{\lambda}{n}W^{(l)},$$

и обновление будет происходить согласно

$$W^{(l)} := W^{(l)} - \frac{\alpha}{n}\left(\frac{\partial J_{\text{nn}}}{\partial W^{(l)}} + \lambda\frac{\partial J_{\text{reg}}}{\partial W^{(l)}}\right),$$

где α — коэффициент скорости обучения и λ — коэффициент регуляризации. Регуляризация смещения не проводится.

Подготовка данных

```
from sklearn import datasets
data = datasets.load_wine()

df = pd.DataFrame(data.data, columns = data.feature_names)
df['target'] = data.target

df = df[df.target != 2]

X = df[['alcohol', 'proline']]
y = df['target']

X = (X - X.mean()) / X.std()

X = X.to_numpy().T
y = y.to_numpy().reshape(1, -1)
```

Обучение модели

Обучим модель.

```
def sigmoid(z):
  s = 1 / (1 + np.exp(-z))
  return s
```

```
np.random.seed(33)

W1 = np.random.randn(3, 2)
b1 = np.random.randn(3, 1)

W2 = np.random.randn(1, 3)
b2 = np.random.randn(1, 1)

n = X.shape[1]

epochs = 100000
learning_rate = 0.1
reg = 0.01

for i in range(epochs):
  # 3 x 2 на 2 x 130
  Z1 = np.dot(W1, X) + b1
  A1 = sigmoid(Z1) # (3 x 130)

  # 1 x 3 на 3 x 130
  Z2 = np.dot(W2, A1) + b2
  A2 = sigmoid(Z2) # (1 x 130)

  loss = objective(A2, y, n, reg)

  W2_delta = A2 - y # (1 x 130)
  W1_delta = np.dot(W2.T, W2_delta) * A1 * (1 - A1) # (3 x 130)

  # 1 x 130 на 130 x 3
  # значение градиента с учетом регуляризации
  W2_derivative = (np.dot(W2_delta, A1.T)  + reg * W2) / n # (1 x 3)
  b2_derivative = np.sum(W2_delta, axis = 1, keepdims = True) / n # (1 x 1)

  # 3 x 130 на 130 x 2
  # значение градиента с учетом регуляризации
  W1_derivative = (np.dot(W1_delta, X.T) + reg * W1) / n # (3 x 2)
  b1_derivative = np.sum(W1_delta, axis = 1, keepdims = True) / n # (3 x 1)

  W2 = W2 - learning_rate * W2_derivative
  b2 = b2 - learning_rate * b2_derivative
  W1 = W1 - learning_rate * W1_derivative
  b1 = b1 - learning_rate * b1_derivative

  if i % (epochs / 5) == 0:
    print('Эпоха:', i)
    print('Ошибка:', loss)
    print('-----------------------')
    time.sleep(0.5)

print('Итоговая ошибка', loss)
print('Нейросеть успешно обучена')
```

```
Эпоха: 0
Ошибка: 10.080406829656138
-----------------------
Эпоха: 20000
Ошибка: 0.9317231212330214
-----------------------
Эпоха: 40000
Ошибка: 0.7472183596959365
```

```
-----------------------
Эпоха: 60000
Ошибка: 0.7116253099095151
-----------------------
Эпоха: 80000
Ошибка: 0.6899824534744711
-----------------------
Итоговая ошибка 0.6746032823864037
Нейросеть успешно обучена
```

Сделаем прогноз.

```
from sklearn.metrics import accuracy_score

Z1 = np.dot(W1, X) + b1
A1 = sigmoid(Z1)
Z2 = np.dot(W2, A1) + b2
A2 = sigmoid(Z2)

y_pred, y_true = A2.flatten() > 0.5, y.flatten()

accuracy_score(y_true, y_pred)
```

```
0.9846153846153847
```

Регуляризация в библиотеке Keras

Приведем регуляризацию модели в библиотеке Keras.

```
import tensorflow as tf
```

```
np.random.seed(42)
tf.random.set_seed(42)

# регуляризация добавляется на каждый слой
l2_reg = tf.keras.regularizers.l2(0.01)

model = tf.keras.models.Sequential([
  tf.keras.layers.Dense(3, activation = 'sigmoid', use_bias = True,
                        kernel_regularizer = l2_reg),

  tf.keras.layers.Dense(1, activation = 'sigmoid', use_bias = True,
                        kernel_regularizer = l2_reg)
])

sgd = tf.keras.optimizers.SGD(learning_rate = 0.1,
                              momentum = 0.,
                              nesterov = False)

model.compile(optimizer = sgd,
              loss = 'binary_crossentropy',
              metrics = ['accuracy'])

model.fit(X.T, y.T,
          epochs = 1000,
          batch_size = 130,
          verbose = 0)

model.summary()
```

```
Model: "sequential_2"
┏━━━
```

```
│ dense_4 (Dense)                      │ (130, 3)                    │              9
├──────────────────────────────────────┼─────────────────────────────┼──────────────
│ dense_5 (Dense)                      │ (130, 1)                    │              4
└──────────────────────────────────────┴─────────────────────────────┴──────────────
 Total params: 15 (64.00 B)
 Trainable params: 13 (52.00 B)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 2 (12.00 B)
```

```
model.history.history['accuracy'][-1]
```

```
0.9538461565971375
```

В целом регуляризаторы в Keras можно задавать следующим образом.

```
# l1, l2 и ElasticNet
tf.keras.regularizers.l1(0.)
tf.keras.regularizers.l2(0.)
tf.keras.regularizers.l1_l2(l1 = 0.01, l2 = 0.01)
```

Подведем итог

На сегодняшнем занятии мы изучили концепцию смещения и разброса модели, рассмотрели принцип L1 и L2 регуляризации и протестировали регуляризацию на линейной регрессии и нейросети. Кроме этого, мы познакомились с алгоритмом координатного спуска.

Дополнительные материалы

Бэггинг для снижения разброса

Нам не обязательно использовать прогноз только одной модели. Можно взять группу или ансамбль (от франц. ensemble — «вместе») моделей. Рассмотрим один из способов ансамблирования, **бэггинг** (bagging), и постараемся с его помощью снизить разброс модели.

Про бэггинг

Бэггинг (bagging) или **b**ootstrap **agg**regat**ing** предполагает три основных шага:

1. **Bootstrapping**. За счет случайных выборок с возвращением (random samples with replacement) создается несколько датасетов на основе исходных данных (subsets). Это означает, что в каждом датасете могут быть как повторяющиеся объекты, так и объекты из других датасетов;
2. **Parallel training**. Затем модели (их называют базовыми моделями, base estimators, или weak learners) обучаются параллельно и независимо друг от друга;
3. **Aggregation**. Наконец, в зависимости от типа задачи, т.е. регрессии или классификации, рассчитывается, соответственно, среднее значение (soft voting) или класс, предсказанный большинством (hard/majority voting).

Алгоритм решающего дерева

Рассмотрение решающих деревьев (decision trees), алгоритма, который мы будем использовать внутри бэггинга, выходит за рамки сегодняшнего занятия.

При этом будет важно сказать, что одним из недостатков этого алгоритма (когда используется только одно дерево) как раз является склонность к переобучению и высокий разброс модели. Посмотрим, сможет ли бэггинг исправить ситуацию.

Подготовка данных

Используем данные о пациентах с диабетом.

diabetes.csv **Скачать**

```
diabetes = pd.read_csv('/content/diabetes.csv')
diabetes.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

```
X = diabetes.drop(columns = 'Outcome')
y = diabetes.Outcome

X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    test_size = 0.3,
                                                    random_state = 42)
```

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth = 10, random_state = 42)
model.fit(X_train, y_train)

y_pred_train = model.predict(X_train)
y_pred_test = model.predict(X_test)

print('train: ' + str(accuracy_score(y_train, y_pred_train)))
print('test: ' + str(accuracy_score(y_test, y_pred_test)))
```

```
train: 0.9795158286778398
test: 0.696969696969697
```

```
from sklearn.ensemble import BaggingClassifier

bag_model = BaggingClassifier(estimator = model,
                              n_estimators = 100,
                              # выборка с возвращением
                              bootstrap = True,
                              random_state = 42)

bag_model.fit(X_train, y_train)

y_pred_train = bag_model.predict(X_train)
```

```python
y_pred_test = bag_model.predict(X_test)

print('train: ' + str(accuracy_score(y_train, y_pred_train)))
print('test: ' + str(accuracy_score(y_test, y_pred_test)))
```

```
train: 1.0
test: 0.7532467532467533
```